Filtered merges make deeper representation trees pay: random Subset Sum in $2^{0.2726n}$

Nick Howgrave-Graham

November 2026

Abstract

For random Subset Sum at density one, the best published heuristic exponent is 0.2830, due to Bonnetain, Bricout, Schrottenloher and Shen (BBSS), using representations with digit set $\{-1, 0, 1, 2\}$. We add type-class (consistency) filters, in the line of Dubiner and of May and Ozerov, to the merges of the BBSS tree. At the BBSS depth the gain is small ($0.2830 \to 0.2793$). However, the filters remove the reason why deeper trees did not help: in the unfiltered tree the upper merges stay the most expensive whatever the depth, whereas filtered upper merges are cheap enough for extra levels to pay. With depth 4, 5 and 6 we obtain 0.2744, 0.2730 and 0.2726, under the standard heuristics. We check the accounting against a survival recursion that every reported optimum satisfies, prove that the filters cannot bias the survival of the planted solution, and test the remaining independence assumptions by simulation.

1 Introduction

An instance of Subset Sum consists of integers $a_1, \ldots, a_n$ and a target t, and asks for $x \in \{0,1\}^n$ with $\langle a, x\rangle = t$. In the *random* setting the a_i are uniform in $[0, 2^n)$ and $t = \langle a, x_0\rangle$ for a planted x_0 of weight $n/2$ (density one). We measure the exponent $\log_2(\text{time})/n$.

Meet-in-the-middle [11, 18] gives $2^{n/2}$. Howgrave-Graham and Joux [12] introduced *representations*: the solution is written in many ways as a sum of two vectors, so a random modular constraint can be imposed on the summands while some representation survives. Becker, Coron and Joux [5] allowed digits -1, and Bonnetain, Bricout, Schrottenloher and Shen [6] added the digit 2 and an accounting in which intermediate lists may be random subsets of their constrained sets. Table 1 summarises.

The story. At each merge of a representation tree, two lists are joined on a modular constraint, and only a small fraction of the matching pairs have the right coordinate pattern. Filtering the pairs by their joint coordinate type, so that inconsistent pairs are never generated, is the consistency technique of May and Ozerov [17], which goes back to Dubiner's bucketing codes [9]. On the BBSS tree at its own depth it gains only about 0.004. BBSS observed [6, Remark 4] that the time "is a decreasing function of the number of levels, which quickly reaches a minimum", because "adding one more level does not change the cost of the upper levels, which will remain the most expensive"; Ozerov makes the same observation for ternary trees [17, §4.1]. Filters change exactly this. They make the upper merges cheap, so additional levels, which shrink the lists, start to pay, and at the optimum the costs of all merges tie. This gives the improvement $0.2830 \to 0.2726$ (Section 5): about 36% of it comes from the filters at fixed depth, and 64% from the depth they make affordable. In decoding the analogous effect is known: with near-neighbour search at every level, BJMM-type trees improve up to depth 4 [3, 4].

exponent	work	digit set	depth	merges
0.337	HGJ [12] (as corrected in [5])	$\{0,1\}$	3	plain
0.329	May–Ozerov [17]	$\{0,1\}$	2	consistency filtered
0.291	BCJ [5]	$\{-1,0,1\}$	3	plain
0.287	May–Ozerov [17]	$\{-1,0,1\}$	3	consistency filtered
0.2852, 0.2814	this work†	$\{-1,0,1\}$	3, 4	type-class filtered
0.2830	BBSS [6]	$\{-1,0,1,2\}$	3	plain
0.2793	this work	$\{-1,0,1,2\}$	3	type-class filtered
0.2744	this work	$\{-1,0,1,2\}$	4	type-class filtered
0.2730	this work	$\{-1,0,1,2\}$	5	type-class filtered
0.2726	this work	$\{-1,0,1,2\}$	6	type-class filtered

Table 1: Heuristic time exponents for random density-one Subset Sum. Depth is the number of levels of representation lists below the root (d in Section 2); BBSS's "five levels, numbered 4 down to 0" [6, §2.5] is $d = 3$ with meet-in-the-middle leaves; our recomputation of [6] gives 0.2831 (Table 2). †BBSS accounting (Heuristic 1); under the strict accounting of [5], which [17] also uses, our filtered depth-3 value is 0.2864, which a depth-4 run under the same accounting did not improve, and Ozerov's accounting gives HGJ 0.340 rather than 0.337, so the rows are not directly comparable.

What is new. Filtering of representation trees is due to May and Ozerov [17], for the HGJ and BCJ digit sets at depth at most 3. Filters combined with depth are known in decoding, from Both and May [3, 4]. New here are filters on the BBSS digit set $\{-1,0,1,2\}$, trees of depth $d \geq 4$, and the soundness analysis.

Contributions.

1. Type-class filtered τ-merges on representation trees with general digit sets, with cost terms in closed form and the observation that the filter law only matters through its label type (Section 3).

2. An optimisation of the BBSS tree with filters at every merge, for depths $d \leq 6$: 0.2744, 0.2730, 0.2726 (Section 5).

3. Soundness of the accounting: a survival recursion that is satisfied with equality by every reported optimum, and a symmetry lemma showing that the planted solution survives each filtered merge with exactly the average probability (Section 4).

4. Experiments testing survival, filter bias and sibling independence (Section 6).

All results are heuristic, in the same sense as [12, 5, 6].

Related work. Consistency filtering for HGJ and BCJ trees is in Ozerov's thesis [17, Ch. 2, §4.2], reporting joint work with May, which leaves digits ± 2 as an open question [17, after Cor. 65]; BBSS list it as "not studied" [6, Table 1]. Böhme [1] reportedly reached 0.2871 with $\{-2,\ldots,2\}$ (unpublished; see [6, fn. 5]). Filtering by joint type is also the mechanism of May–Ozerov near-neighbour search in decoding [16] and of the locality-sensitive filters of Becker, Ducas, Gama and Laarhoven [2]; Christiani [7] gives the asymmetric framework. In the decoding trees of Both and May [3, 4] and the ternary-LWE trees of Kirshanova and May [14] (see [13, App. A] for a

correction to its analysis), near-neighbour search replaces exact matching of values by approximate matching, and the representation coordinates are checked after each merge; our filters act on the representation coordinates and keep exact modular matching. An earlier claim of $2^{0.255n}$ [10] has been withdrawn ("issue with counting duplicate representations"); unlike [10] we never sample or modify list contents, and all representation counts are those of [6]. Li [15] claims $2^{0.24n}$ using a ternary-tree representation structure; we were not able to reproduce this exponent within the heuristics used here. Charging each three-list merge for the pairs it enumerates from two of its lists, rather than for its output size, we obtain 0.39.

2 Preliminaries

Logarithms are to base 2 and H is Shannon entropy. For a finite set Σ and $v \in \Sigma^n$ the *type* of v is its empirical distribution; there are $2^{nH(P)+O(\log n)}$ strings of type P [8]. Exponents are stated up to $O(\log n)/n$.

Digit sets, sets and merges. A *digit set* is a finite $\mathcal{D} \subset \mathbb{Z}$ containing 0 and 1; we use $\{-1, 0, 1, 2\}$ unless stated otherwise. For a modulus m, residue $\tau \in \mathbb{Z}_m$ and type w on $\mathcal{D}$, the (m, τ, w)-*set* is $\{x \in \mathcal{D}^n : \ \mathrm{type}(x) = w, \ \langle a, x\rangle \equiv \tau \pmod m\}$. A τ-*merge* of lists $L_A \subseteq$ an (m', τ_A, w')-set and $L_B \subseteq$ an $(m', \tau - \tau_A, w')$-set, with $m' \mid m$, outputs the $x + y$ ($x \in L_A$, $y \in L_B$) with $\langle a, x + y\rangle \equiv \tau \pmod m$ and $\mathrm{type}(x + y) = w$, i.e. elements of the (m, τ, w)-set. Outputs are kept as a set (no duplicates).

The tree of depth d. Level 0 is the solution x_0, of type $\delta_0 = (\frac{1}{2}, \frac{1}{2})$ on $\{0, 1\}$. Levels $j = 1, \ldots, d$ carry 2^j lists of type-δ_j vectors, constrained modulo $M_j = 2^{c_j n}$ with $1 = c_0 \geq c_1 \geq \cdots \geq c_d \geq 0$, $M_j \mid M_{j-1}$, and residues chosen at random consistently down the tree (the τ's). Each level-$(j-1)$ list is the τ-merge of two level-j lists; the level-d lists are built by meet-in-the-middle from half-vectors, in time $2^{H(\delta_d)n/2}$. The depth d is the number of levels of lists below the root.

Splits. A level-$(j-1)$ digit z is written $z = x + y$ with $x, y \in \mathcal{D}$; the *split law* σ_j is the joint type of (x, y) over all coordinates. It is symmetric, has both marginals δ_j, and is determined by δ_{j-1} and, for each parent digit, a distribution over its unordered splits (for $\{-1, 0, 1, 2\}$: three free fractions, $0 \to (\pm 1, \mp 1)$, $1 \to (2, -1)$, $2 \to (1, 1)$). Write $H_j = H(\delta_j)$. A fixed level-$(j-1)$ vector has $2^{\rho_j n}$ representations of split type σ_j, and a random pair of type-δ_j vectors is such a split with probability $2^{p_j n}$, where

$$\rho_j = H(\sigma_j) - H_{j-1}, \qquad p_j = H(\sigma_j) - 2H_j.$$

Heuristic 1 (BBSS [6])**.** *Every list behaves like a uniformly random subset, of its stated size, of its (M_j, τ, δ_j)-set. The merge of two level-j lists of size $2^{\ell_j n}$ examines $2^{P_j n}$ pairs, $P_j = 2\ell_j - (c_{j-1} - c_j)$, and outputs $2^{(P_j + p_j)n}$ vectors; the expected number of solutions found at the root is at least* 1.

3 Filtered τ-merges

Fix a merge of two level-j lists and drop the index. Let $(X, Y) \sim \sigma$, and let Λ take values in a finite *label set* with conditional law $\kappa(\cdot \mid x, y)$ (one law per ordered pair type). Write Q for the law of (X, Y, Λ) and Q_Λ for its label marginal.

Definition 1. A *filter* is a string u in the label set, of type Q_Λ. A left vector x *enters* u if the joint type of (x,u) is the (X,Λ)-marginal of Q, and a right vector y enters u if the joint type of (y,u) is the (Y,Λ)-marginal. The filtered τ-merge draws $F = 2^{I(XY;\Lambda)n}$ random filters and, within each, matches the entering left and right vectors on the new constraint bits.

These are Dubiner's random bucketing codes [9], specialised to a representation merge; with a two-symbol label set and recursive block structure they are Ozerov's consistency algorithm [17, §2.2].

Lemma 2. *Up to $2^{O(\log n)}$: a pair of joint type σ enters a common random filter with probability $2^{-I(XY;\Lambda)n}$; a left (right) vector enters with probability $2^{-I(X;\Lambda)n}$ ($2^{-I(Y;\Lambda)n}$); a pair of independent type-δ vectors enters a common filter with probability $2^{-(I(X;\Lambda)+I(Y;\Lambda))n}$.*

Proof. Method of types, as in [9]: for fixed (x,y) of type σ, $2^{nH(\Lambda|XY)}$ of the $2^{nH(\Lambda)}$ strings of type Q_Λ have joint type Q with it; similarly for single vectors; for independent x,y the two events are independent given u. □

Heuristic 2. *The pairs that agree on the constraint bits behave, with respect to filters, like independent uniform pairs of type-δ vectors.*

Proposition 3. *Under Heuristics 1 and 2, the filtered τ-merge outputs $2^{(P+p)n}$ vectors up to a constant factor, at cost dominated by*

$$\underbrace{\ell + I(XY;\Lambda) - I(X;\Lambda)}_{\text{entries (left)}}, \quad \underbrace{\ell + I(XY;\Lambda) - I(Y;\Lambda)}_{\text{entries (right)}}, \quad \underbrace{P + I(XY;\Lambda) - I(X;\Lambda) - I(Y;\Lambda)}_{\text{pairs passing}},$$

plus the cost of decoding, for each vector, the filters it enters.

With $F = 2^{I(XY;\Lambda)n}$ filters a valid pair shares $\Theta(1)$ filters in expectation, so a constant fraction of valid pairs could be missed; we use nF filters, which makes the miss probability $e^{-\Omega(n)}$ per pair at a polynomial cost. A filter pays when $I(X;\Lambda)+I(Y;\Lambda) > I(XY;\Lambda)$; we call $I(X;\Lambda)+I(Y;\Lambda)-I(XY;\Lambda)$ the *saving* of the merge.

Remark 4 (Only the label type matters). One may draw filters i.i.d. from any law π on the label set; the number of filters and the entry probabilities then change by factors $2^{\pm D(Q_\Lambda\|\pi)n}$ that cancel in all three cost terms, since only filters of type Q_Λ ever contain a valid pair. We therefore normalise $\pi = Q_\Lambda$ (see also Appendix A).

Decoding the filters. Each vector must find the $2^{(I(XY;\Lambda)-I(X;\Lambda))n}$ filters it enters, among $2^{I(XY;\Lambda)n}$, in time close to that number. Random codes do not allow this, so we use a product construction, as in Dubiner's bucketing codes [9] and the tensored filters of [2]; it does not need the code rates to be small.

Proposition 5. *The filtered τ-merge can be implemented so that each cost term of Proposition 3 is achieved up to a factor $2^{o(n)}$, including the time to decode the filters, while each valid pair of type σ is still found with probability $1-o(1)$.*

Proof sketch. Apply a uniformly random permutation to the coordinates (the instance is exchangeable) and cut them into $b = \lceil\sqrt{n}\rceil$ blocks of size about $\sqrt{n}$. For a valid pair, the vector of its per-block joint types, its *pattern*, is a sequence $(\sigma_1,\dots,\sigma_b)$ averaging to σ. By concentration of sampling without replacement, with probability $1-o(1)$ every σ_i is within $\epsilon_n = n^{-1/4}\log n$ of σ in total variation. The number of such patterns is at most $(\sqrt{n}+1)^{O(b)} = 2^{O(\sqrt{n}\log n)}$.

For each pattern we build an independent filter family: in block i, a random code C_i of $2^{\sqrt{n}\, I_{\sigma_i}(XY;\Lambda)}$ label strings of type $Q_{\Lambda,i}$ (the label marginal of $\sigma_i \times \kappa$), and a filter is a tuple $(u_1, \ldots, u_b) \in C_1 \times \cdots \times C_b$. A left vector enters the tuple if it enters every u_i in the sense of Section 3, with block-i marginals; similarly on the right. A vector can only enter a tuple whose block marginals equal its own block types, so it enters only the family of its own pattern, and a vector with an atypical pattern enters no family at all: atypical vectors add nothing to the entry cost. For a fixed vector, a Chernoff bound per block gives an atypical pattern with probability $2^{-\Theta(\log^2 n)}$, so the valid pairs lost in this way are an $o(1)$ fraction. A vector decodes its filters by testing, in each block, all of C_i (time $2^{O(\sqrt{n})}$) and then enumerating the product of the per-block lists, in time proportional to the output plus $b\, 2^{O(\sqrt{n})}$.

Lemma 2 applies in each block. The probabilities multiply, so a valid pair with this pattern shares $\Theta(1)$ filters in expectation (amplified as above), and the entry and passing exponents are block averages of the single-block terms. Those terms are continuous functions of the pair type for fixed κ, so each average differs from the global term of Proposition 3 by $O(\epsilon_n)$, i.e. by $o(n)$ in the exponent. Summing over the $2^{o(n)}$ patterns multiplies all costs by $2^{o(n)}$. □

4 Accounting and soundness

The program. For fixed split laws and filter laws, the exponents (c_j, ℓ_j, T) are the variables of a linear program: minimise T subject to Heuristic 1 ($\ell_{j-1} = P_j + p_j$, $\ell_j + c_j \le H_j$, root yield ≥ 1), nesting $c_j \le c_{j-1}$, and T at least the leaf build $H_d/2$, every list size, and, at every merge, the three terms of Proposition 3 (or P_j if unfiltered). The split and filter laws are optimised by repeated Powell and Nelder–Mead searches, warm-started from shallower optima.

Survival recursion. Heuristic 1 allows $c_j > \rho_j$ at a level (fewer representations than the constraint removes), which is not obviously sound: a level-$(j-1)$ element then survives only with probability $2^{-(c_j - \rho_j)n}$. Let $s_j \le 0$ be the log-survival exponent of a level-j element, with $s_d = 0$ (complete leaves). Saturating the Poisson survival law $\sigma = 1 - e^{-c\sigma_A\sigma_B}$ gives

$$s_j = \min\bigl(0,\ \rho_{j+1} - c_{j+1} + 2s_{j+1}\bigr) \quad (j = d-1, \ldots, 1), \qquad \rho_1 - c_1 + 2s_1 \ge 0 \text{ (root)},$$

with list sizes $\ell_j = H_j - c_j + s_j$. Excess representations at a split compensate deficits deeper in the tree (closer to the leaves); since survival is capped at 1, they cannot be carried upward to pay for a deficit closer to the root. So the accounting of Heuristic 1 is sound exactly when every deficit lies below a sufficient excess. We solve the program with these constraints in place of Heuristic 1 (exactly, by enumerating which levels saturate). **Every reported optimum gives the same value under both accountings.** At depth 4, only the split into level 2 has a deficit ($s_1 = -0.059$), paid exactly by the root excess $\rho_1 - c_1 = 0.119 = -2s_1$ (Table 3).

Symmetry. Filtered lists are not uniform random subsets in any obvious sense, but the planted solution is not disadvantaged.

Lemma 6. *Fix a node (per node), its exact type, and the randomness: the instance a with i.i.d. uniform coordinates, the node's residue τ, marginally uniform and independent of a, the residues below it (functions of τ and fresh independent randomness), and i.i.d. filters independent of all of these. Let O be the output list and V_τ the parents of the exact type in the node's residue class. Then for every parent P of that type,*

$$\Pr[P \in O \mid P \in V_\tau] = \mathbb{E}|O| \,/\, \mathbb{E}|V_\tau|,$$

and the same holds at every level.

Proof. Every step (complete coset leaves, modular joins, validity checks, the exact-type output restriction, and filter entry, which depends only on joint (digit, label) counts) is equivariant under coordinate permutations g: $O(ga, \tau, gu) = g\,O(a, \tau, u)$. The law of (a, τ, u) is g-invariant and g maps V_τ to itself, so $\Pr[P \in O \wedge P \in V_\tau]$ is the same for all P of the type, on which the permutations act transitively. Summing over P and dividing by $\Pr[P \in V_\tau] = 1/M$ gives the claim; child lists are outputs of equivariant steps, so induction over levels applies.

The hypothesis on residues holds in the planted model. The root target $t = \langle a, x_0 \rangle$ depends on a, but at every node the residue is τ or $t - \tau$ (reduced modulo the node's modulus) for a τ that is uniform and independent of (a, x_0), so each node residue is marginally uniform and independent of a. The planted parent P^* at a node is a function of x_0 and of the split choices, so it is independent of a, and the lemma applies to it. □

The lemma concerns the ratio of expectations, which is the quantity the accounting uses; it says nothing about joint survival of two nodes. In particular sibling residues τ and $t - \tau$ are dependent through t; this joint dependence is what the sibling-independence experiment of Section 6 tests.

5 Results

digit set	merges	$d = 1$	2	3	4	5	6
$\{0, 1\}$	plain	0.4056	0.3371	0.3371	0.3371	0.3371	0.3371
	filtered	0.4056	0.3269	0.3269	0.3269	0.3269	0.3269
$\{-1, 0, 1\}$	plain	0.4056	0.3122	0.2891	0.2891	0.2891	0.2891
	filtered	0.4056	0.3122	0.2852	0.2814	0.2814	0.2814
$\{-1, 0, 1, 2\}$	plain	0.4056	0.3025	0.2831	0.2831	0.2831	0.2831
	filtered	0.4056	0.3025	0.2793	0.2744	0.2730	0.2726

Table 2: Time exponents by depth d, BBSS accounting, plain and filtered merges (each entry is the best found; a deeper tree can copy a shallower one, so entries are monotone). $\{0, 1\}$ and $\{-1, 0, 1\}$ under BBSS accounting give 0.3371 and 0.2891; the strict accounting of [12, 5] gives 0.3371 and 0.2909.

Theorem 7 (heuristic)**.** *Under Heuristics 1 and 2, random density-one Subset Sum can be solved in time* $2^{(0.2726+o(1))n}$ *with a filtered* $\{-1, 0, 1, 2\}$ *tree of depth* 6*, and in time* $2^{(0.2744+o(1))n}$ *with depth* 4*. The* $o(1)$ *comes from filter decoding (Proposition 5).*

Structure of the optimum. Table 3 gives the depth-4 point. The filtered passing costs of all four merges tie at T. At this point filtering is active only at the dense levels 1–2, with savings 0.076 and 0.048; at levels 3–4 the optimal filter is essentially trivial ($I(XY; \Lambda) \leq 0.022$). The deficit at level 2 is paid by the root excess, and the survival recursion is tight. The deeper optima (Tables 5 and 6) filter at the dense top levels *and* at the sparse bottom level ($I(XY; \Lambda) = 0.21$ and 0.28 at the last merge for $d = 5, 6$). There the lists are mostly zeros, and at $d = 6$ the bottom level has $\delta_6(1) = 0.0003$ against $\delta_6(2) = 0.0137$ and $\delta_6(-1) = 0.0199$: it consists of 2s and -1s split from the level-5 ones by $1 \to (2, -1)$. Lemma 6 holds at every level, sparse ones included, so these merges

need no separate bias argument (see also Section 6). The gain over BBSS, 0.0105, splits into 0.0038 from filters at depth 3 and 0.0067 from the extra depth.

j	$\delta_j(-1)$	$\delta_j(0)$	$\delta_j(1)$	$\delta_j(2)$	ρ_j	c_j	s_j	ℓ_j
1	0.0482	0.6592	0.2870	0.0056	0.9917	0.8732	-0.0593	0.2334
2	0.0561	0.7635	0.1797	0.0007	0.7099	0.7690	-0.0001	0.2134
3	0.0491	0.8398	0.1106	0.0005	0.5165	0.5166	0.0000	0.2650
4	0.0291	0.9105	0.0603	0.0000	0.2422	0.2422	0.0000	0.2744

merge of level j	pairs	$I(XY;\Lambda)$	entries gap	saving	passing
1	0.3399	0.1584	0.0410	-0.0764	0.2635
2	0.3227	0.1702	0.0610	-0.0483	0.2744
3	0.2776	0.0220	0.0094	-0.0032	0.2744
4	0.2744	0.0000	0.0000	0.0000	0.2744

leaves $H_4/2 = 0.2583$; root excess $\rho_1 - c_1 = +0.1185$, $2s_1 = -0.1185$; $T = 0.27440$

Table 3: The depth-4 optimum: levels (top) and merges (bottom). Columns: $\delta_j(\cdot)$, the digit type of level j; ρ_j, the representation exponent of a level-$(j-1)$ vector into level-j pairs; c_j, the constraint bits ($M_j = 2^{c_j n}$); s_j, the survival exponent of Section 4; ℓ_j, the list size; *pairs*, $P_j = 2\ell_j - (c_{j-1} - c_j)$, the pairs an unfiltered merge would examine; $I(XY;\Lambda)$, the code rate (at $\pi = Q_\Lambda$); *entries gap*, $I(XY;\Lambda) - \min(I(X;\Lambda), I(Y;\Lambda))$, the log number of filters per vector; *saving*, $I(XY;\Lambda) - I(X;\Lambda) - I(Y;\Lambda)$ (negative means fewer pairs); *passing*, pairs + saving, the pairs the filtered merge examines. All exponents are per n.

Saturation. The exponent saturates with depth: 0.2793, 0.2744, 0.2730, 0.2726 for $d = 3, \ldots, 6$. [TODO: d=6 polish still running; d=7?]

Wider digit sets. Adding the digit -2 gains at most 0.0004 at fixed depth (0.2744 → 0.2740 at $d = 4$, 0.2730 → 0.2727 at $d = 5$, and 0.0001 at $d = 6$ against a $\{-1, 0, 1, 2\}$ re-optimisation from the same start); the optimiser keeps a small positive -2 mass. The digit 3 has zero marginal value, since it can only arise from splitting a 2, and an optimisation over $\{-4, \ldots, 4\}$ did not converge and was stopped. We therefore keep the BBSS digit set $\{-1, 0, 1, 2\}$ throughout.

Label sets. Re-optimising the filters at depth 5 with a 4-symbol label set, warm-started from the 3-symbol optimum, gives 0.273224 against 0.273335: a gain of 0.0001, so 3-symbol filters appear essentially optimal.

6 Experimental validation

Survival. Table 4: a depth-3 HGJ tree (weights $n/2 \to n/4 \to n/8 \to n/16$) with complete leaves (leaf ratio $c \approx 3.9$) and internal ratio $c \in \{1, 2, 3, 4\}$. The Poisson model $\sigma = 1 - e^{-c\sigma_A\sigma_B}$ is slightly optimistic at every level, but survival at $c = 3$ does not decay, and the work per success is flat in c to within about one bit (conservation: raising c buys nothing).

Filter bias. A filtered merge at $n = 32$ shaped like the level-2 merge of the optimum (complete coset lists of about 1900 elements, a 3-symbol label set from the optimum's conditionals, 300

n (draws)	c	σ_2	Poisson	σ_1	Poisson	Pr[x_0 found] (Poisson)	$\log_2$ work/success
48 (40)	1	0.953	0.982	0.593	0.618	0.15 (0.32)	21.3
	2	0.954	0.982	0.823	0.856	0.65 (0.77)	20.5
	3	0.943	0.982	0.906	0.943	0.98 (0.93)	20.8
	4	0.957	0.982	0.957	0.979	1.00 (0.98)	21.4
64 (10)	1	0.963	0.980	0.602	0.617	0.30 (0.32)	25.3
	2	0.967	0.980	0.841	0.853	0.50 (0.77)	26.0
	3	0.953	0.980	0.928	0.944	0.90 (0.93)	26.1
	4	0.964	0.980	0.972	0.978	1.00 (0.98)	26.6

Table 4: Survival by level in a real HGJ tree (`floor/survival.py`). $\sigma_j = |L_j|/\mathbb{E}|V_j|$. At $n = 64$ the modulus is capped at 2^{56} (density 1.14).

draws): the planted parent survives with probability 0.8633 against mean $\sigma = 0.8638$ without filters, and 0.7200 ± 0.0259 against 0.7150 with filters, as Lemma 6 predicts. A second merge at $n = 20$ is shaped like the sparse bottom merge of the depth-6 optimum: the parent has four 1s and two -1s, and the children have two 2s and three -1s, joined by $1 \to (2, -1)$ and $-1 \to (-1, 0)$ (`floor/bias_sim_sparse.py`, 400 draws). The planted parent survives with probability 0.598 ± 0.025 against $\sigma = 0.611$ without filters, and 0.438 ± 0.025 against 0.445 with filters.

Sibling independence. Two planted siblings (disjoint supports, shared instance, independent residues), each built by its own filtered merge, 4000 draws per n: $\Delta = \log_2 \Pr[\text{both}] - \log_2 \Pr[A] - \log_2 \Pr[B]$ is $+0.0005$, $+0.0091$, -0.0048, $+0.0077$ at $n = 24, 32, 40, 48$ with one shared code (similar with independent codes and without filters), all within ± 0.02 and with no trend in n. So the exactly tight root condition needs no extra excess.

7 Conclusion and open problems

Filters and depth attack the same waste from two sides, and together they give 0.2726. The exponent appears to saturate near 0.272. Open: a full proof of the concentration step in Proposition 5; non-binary roots and non-enumerative top levels; wider digit sets, which add at most about 0.0004 at fixed depth; and transferring the observation to decoding and ternary-LWE trees, where merges also generate pairs with the wrong split pattern. Lattice-based hybrids, such as guessing combined with batch-CVP, are competitive at density one; whether they can be combined with filtered representation trees is open.

A Parameters and reproduction

In the numerical model the filter law π is a free parameter; at one optimum it drifted to $\pi_N \approx 0$, giving formal code rates of $34n$ and $128n$ at two levels, while all costs were unchanged on resetting $\pi = Q_\Lambda$ (Remark 4). The rates in the tables are at $\pi = Q_\Lambda$.

Tables 5 and 6 give the depth-5 and depth-6 points (columns as in Table 3). Points: `floor/runs/polish/gen_m4_5_f_11.npy`, `floor/runs/polish/gen_m4_6_f_11.npy`. All numbers in this draft are generated by `floor/make_numbers.py`; the model is `gen_lsf.py`, the optimiser `gen_opt.py`, and the survival-recursion accounting `floor/recursive_floor.py` (variants S1/S2). Commit: [TODO: hash].

j	$\delta_j(-1)$	$\delta_j(0)$	$\delta_j(1)$	$\delta_j(2)$	ρ_j	c_j	s_j	ℓ_j
1	0.0445	0.6675	0.2814	0.0066	0.9708	0.8530	-0.0589	0.2396
2	0.0463	0.7844	0.1675	0.0019	0.6364	0.6953	0.0000	0.2335
3	0.0387	0.8602	0.1008	0.0002	0.4362	0.4362	0.0000	0.2686
4	0.0215	0.9257	0.0528	0.0000	0.1834	0.1702	-0.0066	0.2697
5	0.0247	0.9489	0.0125	0.0139	0.1270	0.1336	0.0000	0.2349

merge of level j	pairs	$I(XY;\Lambda)$	entries gap	saving	passing
1	0.3323	0.1142	0.0269	-0.0604	0.2718
2	0.3093	0.1153	0.0395	-0.0362	0.2730
3	0.2782	0.0140	0.0044	-0.0051	0.2730
4	0.2734	0.0070	0.0033	-0.0004	0.2730
5	0.4332	0.2115	0.0244	-0.1626	0.2706

leaves $H_5/2 = 0.1842$; root excess $\rho_1 - c_1 = +0.1177$, $2s_1 = -0.1177$; $T = 0.27305$

Table 5: The depth-5 point.

References

[1] E. Böhme, *Verbesserte Subset-Sum Algorithmen*, Master's thesis, Ruhr-Universität Bochum, 2011.

[2] A. Becker, L. Ducas, N. Gama and T. Laarhoven, New directions in nearest neighbor searching with applications to lattice sieving, in *Proc. SODA 2016*, SIAM, pp. 10–24.

[3] L. Both and A. May, Optimizing BJMM with nearest neighbors: full decoding in $2^{2n/21}$ and McEliece security, *International Workshop on Coding and Cryptography (WCC 2017)*, 2017.

[4] L. Both and A. May, Decoding linear codes with high error rate and its impact for LPN security, in *PQCrypto 2018*, LNCS, Springer, pp. 25–46. IACR ePrint 2017/1139.

[5] A. Becker, J.-S. Coron and A. Joux, Improved generic algorithms for hard knapsacks, in *EUROCRYPT 2011*, LNCS 6632, Springer, pp. 364–385.

[6] X. Bonnetain, R. Bricout, A. Schrottenloher and Y. Shen, Improved classical and quantum algorithms for subset-sum, in *ASIACRYPT 2020*, LNCS, Springer. IACR ePrint 2020/168, arXiv:2002.05276.

[7] T. Christiani, A framework for similarity search with space-time tradeoffs using locality-sensitive filtering, in *Proc. SODA 2017*, SIAM, pp. 31–46.

[8] I. Csiszár and J. Körner, *Information Theory: Coding Theorems for Discrete Memoryless Systems*, 2nd ed., Cambridge University Press, 2011.

[9] M. Dubiner, Bucketing coding and information theory for the statistical high-dimensional nearest-neighbor problem, *IEEE Trans. Inf. Theory* **56** (2010), no. 8, 4166–4179.

[10] A. Esser and A. May, Better sample – random subset sum in $2^{0.255n}$ and its impact on decoding random linear codes, arXiv:1907.04295 (withdrawn).

[11] E. Horowitz and S. Sahni, Computing partitions with applications to the knapsack problem, *J. ACM* **21** (1974), 277–292.

j	$\delta_j(-1)$	$\delta_j(0)$	$\delta_j(1)$	$\delta_j(2)$	ρ_j	c_j	s_j	ℓ_j
1	0.0423	0.6708	0.2814	0.0055	0.9514	0.8527	-0.0493	0.2333
2	0.0476	0.7813	0.1696	0.0015	0.6553	0.7046	0.0000	0.2307
3	0.0402	0.8574	0.1021	0.0003	0.4488	0.4488	0.0000	0.2673
4	0.0226	0.9237	0.0538	0.0000	0.1913	0.1801	-0.0056	0.2706
5	0.0127	0.9599	0.0266	0.0008	0.1003	0.0672	-0.0194	0.1977
6	0.0199	0.9660	0.0003	0.0137	0.0478	0.0672	0.0000	0.1819

merge of level j	pairs	$I(XY;\Lambda)$	entries gap	saving	passing
1	0.3194	0.0976	0.0233	-0.0509	0.2685
2	0.3132	0.1244	0.0419	-0.0406	0.2726
3	0.2787	0.0167	0.0053	-0.0061	0.2726
4	0.2726	0.0012	0.0006	-0.0001	0.2725
5	0.2825	0.0302	0.0101	-0.0099	0.2726
6	0.3638	0.2829	0.0625	-0.1579	0.2059

leaves $H_6/2 = 0.1246$; root excess $\rho_1 - c_1 = +0.0987$, $2s_1 = -0.0987$; $T = 0.27257$

Table 6: The depth-6 point.

[12] N. Howgrave-Graham and A. Joux, New generic algorithms for hard knapsacks, in *EUROCRYPT 2010*, LNCS 6110, Springer, pp. 235–256.

[13] M. Hhan, J. Kim, C. Lee and Y. Son, A hybrid of lattice-reduction and Meet-LWE via near-collision on Babai's plane, *Des. Codes Cryptogr.* **94** (2026), no. 4, art. 83. IACR ePrint 2022/1473.

[14] E. Kirshanova and A. May, How to find ternary LWE keys using locality sensitive hashing, in *IMACC 2021*, LNCS, Springer. IACR ePrint 2021/1255.

[15] Y. Li, From subset-sum to decoding: improved classical and quantum algorithms via ternary representation technique, *Information* **16** (2025), no. 10, 887.

[16] A. May and I. Ozerov, On computing nearest neighbors with applications to decoding of binary linear codes, in *EUROCRYPT 2015*, LNCS 9056, Springer, pp. 203–228.

[17] I. Ozerov, *Combinatorial Algorithms for Subset Sum Problems*, PhD thesis, Ruhr-Universität Bochum, 2016.

[18] R. Schroeppel and A. Shamir, A $T = O(2^{n/2})$, $S = O(2^{n/4})$ algorithm for certain NP-complete problems, *SIAM J. Comput.* **10** (1981), 456–464.